

Curious Jack

A User Guide

Paul Fishwick and Claude Code

February 2026

Contents

1	Introduction	2
2	Getting Started	2
2.1	Starting the Server	2
3	API Key Configuration	2
3.1	Environment Variable (Recommended)	2
3.2	In-Browser Modal	2
4	The Interface	3
4.1	Parameters Bar	3
4.2	Uploading an Image	3
4.3	Entering Questions	3
5	Results	4
5.1	Multi-Model Synthesis	4
5.2	Image Generation	5
6	Export	5
7	Security	6
7.1	DOMPurify Sanitization	6
7.2	Content Security Policy	6
7.3	HTML Escaping	7
8	Technical Notes	7
9	References and Inspirations	7

1 Introduction

Curiosity is a web application that uses multiple AI models to answer questions about an image. A user uploads a photograph or artwork, selects educational parameters, and types one or more questions. The application sends each question — together with the image — to three large language models (Google Gemini, OpenAI GPT, and Anthropic Claude) running in parallel via the OpenRouter API. A coordinator model then synthesizes the three responses into a single, level-appropriate answer and aligns it to the relevant educational standards for the chosen US state.

2 Getting Started

2.1 Starting the Server

Curiosity runs as a local Node.js server. From the project directory, run the included start script:

```
./start
```

The server starts on `http://localhost:3456`. Open that URL in your browser to use the application.

To stop the server, run:

```
./stop
```

3 API Key Configuration

Curiosity requires an [OpenRouter](#) API key to call the language models. There are two ways to provide it.

3.1 Environment Variable (Recommended)

Set `OPENROUTER_API_KEY` in your terminal before starting the server, or add it permanently to `~/.env`:

```
# Temporary (current terminal session only)
export OPENROUTER_API_KEY=sk-or-...
./start
```

```
# Permanent (add to ~/.env)
OPENROUTER_API_KEY=sk-or-...
```

When the key is found at startup, the application skips the modal entirely and the key never touches the browser or local storage — the most secure option.

3.2 In-Browser Modal

If no environment variable is set, the application checks the browser's `localStorage` for a previously saved key. If none is found, the API Key modal appears, as shown in [Figure 1](#). Enter your OpenRouter key and click **Save & Continue**. The key is saved to `localStorage` so it persists across page reloads.

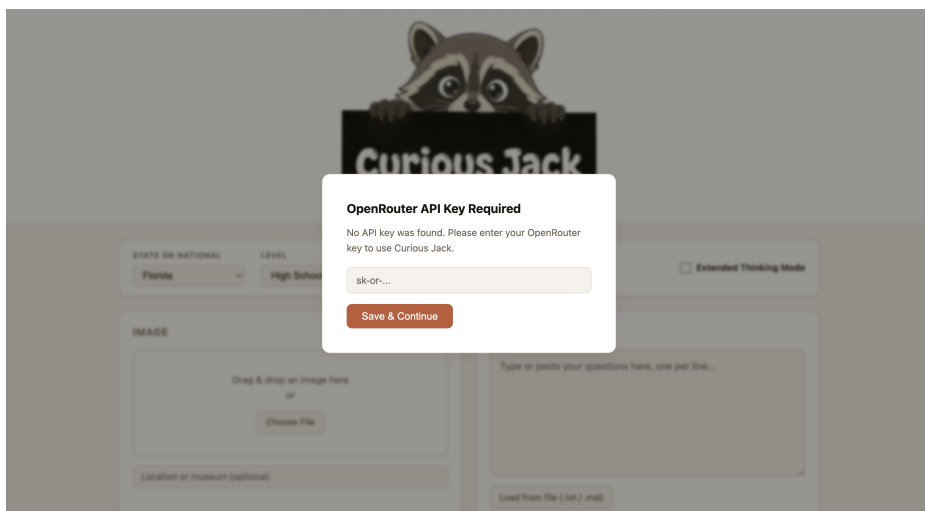


Figure 1: The API key modal appears on first use when no server-side key is configured. The key is stored in `localStorage` for subsequent visits.

4 The Interface

The main interface, shown in Figure 2, consists of a parameters bar at the top and two input cards below it — one for the image and one for the questions.

4.1 Parameters Bar

Four parameters shape the AI response:

- **State** — the US state whose educational standards are cited in the results (e.g. Florida, Texas, National).
- **Level** — the academic level of the answer: Elementary, Middle, High School, or Advanced / College.
- **Detail** — Low (one subject, concise), Medium (two or three subjects), or High (four or more subjects, comprehensive).
- **Extended Thinking Mode** — when checked, each model uses its extended reasoning capability (Claude uses `thinking` blocks, Gemini uses the `:thinking` variant, GPT uses `reasoning_effort: high`).

4.2 Uploading an Image

Drag and drop an image onto the left card or click **Choose File** to open a file picker. Once loaded, a preview of the image replaces the drop zone. An optional **Location** or **museum** field lets you note where the image was taken or sourced; this context is passed to the models and included in the exported report.

4.3 Entering Questions

Type one question per line in the right card. You may also click **Load from file** to populate the textarea from a `.txt` or `.md` file. Blank lines and Markdown headers are ignored, so you can paste

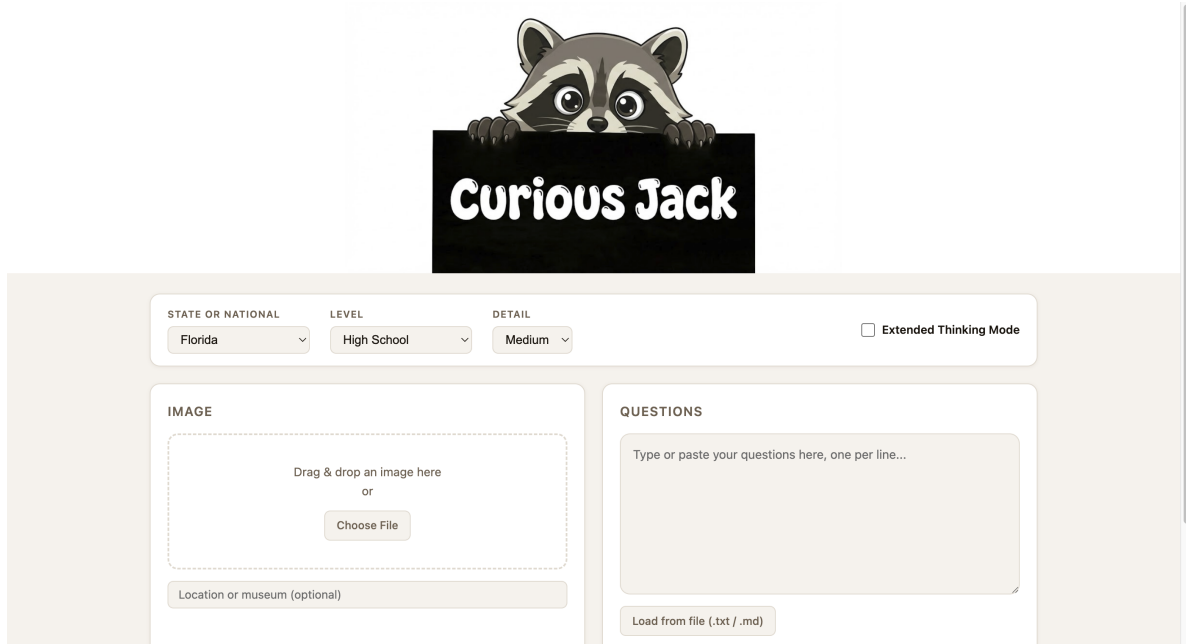


Figure 2: The main Curiosity interface. The parameters bar runs across the top; the image upload card is on the left and the questions card is on the right.

a structured notes file directly. Figure 3 shows the application with an image loaded and questions ready.

5 Results

Clicking Ask processes each question in turn. A progress bar and label track the current question number. As each question completes, its answer card appears and scrolls into view. A sample results card is shown in Figure 4.

Each card contains:

- The question number and the original question text.
- **Subject tags** — the academic disciplines identified by the coordinator model (e.g. Art History, Physics, Chemistry).
- **Answer** — a synthesized, markdown-formatted explanation that notes where the three worker models agreed and where they differed.
- **Educational Standards** — one to three standards from the chosen state that are directly relevant to the question, with standard codes and links to the official standards pages.

5.1 Multi-Model Synthesis

Internally, each question is sent to three worker models in parallel:

The coordinator is instructed to: state majority-agreed facts confidently, acknowledge genuine disagreements, incorporate unique insights from individual workers where plausible, and align the final answer to the chosen educational standards.

STATE OR NATIONAL

LEVEL

DETAIL

Extended Thinking Mode

Florida

High School

Medium

IMAGE

Museo del Prado, Madrid

QUESTIONS

What does the composition and lighting tell us about the artist's technique?

What subjects does this image connect to?

Load from file (.txt / .md)

Ask

Curious Jack — Paul Fishwick and Claude Code

Figure 3: The application with an image uploaded and questions entered, ready to submit. The image preview replaces the drop zone and a location note has been added.

Worker	Model (via OpenRouter)
Gemini	google/gemini-3.1-pro-preview
OpenAI	openai/gpt-5.2
Claude	anthropic/claude-opus-4-6
Coordinator	anthropic/claude-opus-4-6

Table 1: The four model roles used per question. The coordinator synthesizes the three worker responses.

5.2 Image Generation

If a question asks the AI to *create* or *generate* an image (e.g. “Draw a diagram of...”), the coordinator routes the request to Google Gemini’s image generation model (**gemini-3-pro-image-preview**) rather than returning a text answer. The generated image is displayed in the results card with a download link.

6 Export

After all questions are answered, two export buttons appear below the results:

- **Export Markdown** — downloads a .md file containing the full report: parameters, subject image (embedded as base64), all questions, answers, and standards.
- **Export PDF** — opens the browser’s print dialog. The print stylesheet hides all controls and displays only the banner, parameters, image, and answer cards, producing a clean PDF.

Sessions are also auto-saved to the server under `images/<label>/session.json` (alongside a copy

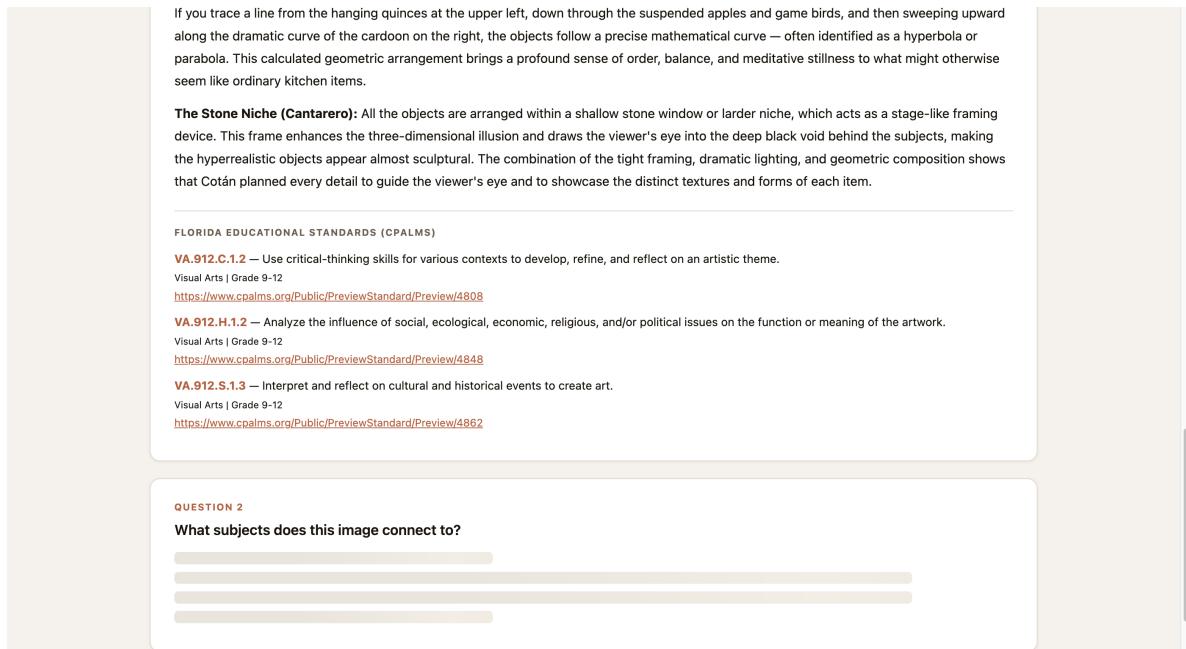


Figure 4: A completed answer card showing subject tags, the synthesized answer, and aligned Florida educational standards with CPALMS codes and links.

of the image), where `<label>` is a short descriptive name generated by the coordinator model from the uploaded image.

7 Security

Because the API key can reside in the browser's `localStorage`, the frontend includes several layers of protection against cross-site scripting (XSS) attacks that could attempt to read and exfiltrate it.

7.1 DOMPurify Sanitization

AI model responses are converted from Markdown to HTML using the `marked` library and then injected into the DOM via `innerHTML`. A malicious response could embed `<script>` tags or `onerror` handlers designed to steal the stored key. All `marked.parse()` output is therefore passed through `DOMPurify` before touching the DOM:

```
DOMPurify.sanitize(marked.parse(answer.answer || ''))
```

DOMPurify strips dangerous tags and event attributes while preserving safe formatting.

7.2 Content Security Policy

A `Content-Security-Policy` meta tag instructs the browser to refuse any script not loaded from an approved source, and to block outbound connections to any host other than `localhost`:

```
default-src 'self';
script-src 'self' https://cdn.jsdelivr.net
          https://cdnjs.cloudflare.com;
style-src 'self' 'unsafe-inline';
```

```
img-src 'self' data: blob;;
connect-src 'self';
```

Even if an XSS payload bypassed DOMPurify, the browser would block it from executing or phoning home.

7.3 HTML Escaping

All user-supplied text (questions, location field, subject tags, standards text) is passed through an `escHtml()` function before being inserted into `innerHTML` template strings. This prevents user-crafted input from being interpreted as HTML.

8 Technical Notes

- The server is a lightweight Express.js application (`server.js`) that proxies all model calls server-side, keeping the API key out of browser network traffic when using the environment-variable method.
- The frontend (`index.html` + `app.js`) is a static single-page application with no build step required.
- `dotenv` is configured to load `~/.env` automatically, so a key placed there is available to all projects that use the same pattern.
- The server runs on port **3456**.

9 References and Inspirations

The following works informed the design and pedagogical philosophy of Curious Jack — in particular the ideas of slow, attentive looking, patient observation, and curiosity-driven inquiry.

- **Slow Art Day** — slowartday.com. An annual global event in which participants spend extended time looking at a small number of works of art, followed by group discussion. Curious Jack encourages the same unhurried attention before submitting questions.
- Shari Tishman, *Slow Looking*. [Harvard Education Press](https://www.harvard.edu/publishing). Tishman’s research on slow, deliberate looking as a vehicle for deep learning across disciplines underpins the observation prompt shown in the image lightbox.
- Rob Walker, *The Art of Noticing*. robwalker.net. A practical guide to paying closer attention to the world, with exercises for sharpening perception and curiosity — the spirit Curious Jack tries to cultivate through structured questioning.
- Alexandra Horowitz, *On Looking: Eleven Walks with Expert Eyes*. alexandrahorowitz.net. Horowitz revisits a single city block with eleven different experts, showing how knowledge and attention transform what we see. The multi-model synthesis in Curious Jack echoes this multi-perspective approach to observation.
- **New York Times 10-Minute Challenge** — [nytimes.com/spotlight/10-minute-challenge](https://www.nytimes.com/spotlight/10-minute-challenge). A collection of short, timed exercises in close reading and observation. The lightbox prompt in Curious Jack (“Take your time and look carefully...”) shares the same intention.